

```
1 interface Model {
2     String getModelName();
3 }
4
5 interface VehicleType {
6     String getTypeName();
7 }
8
9 class SedanModel implements Model {
10     @Override
11     public String getModelName() {
12         return "Sedan";
13     }
14 }
15
16 class SportModel implements Model {
17     @Override
18     public String getModelName() {
19         return "Sport";
20     }
21 }
22
23 class FamilyType implements VehicleType {
24     @Override
25     public String getTypeName() {
26         return "Family";
27     }
28 }
29
30 class RacingType implements VehicleType {
31     @Override
32     public String getTypeName() {
33         return "Racing";
34     }
35 }
36
37 abstract class AbstractFactory {
38     abstract Model createModel();
39
40     abstract VehicleType createType();
41 }
42
43 class CarFactory extends AbstractFactory {
44     @Override
45     Model createModel() {
46         return new SedanModel();
47     }
48
49     @Override
50     VehicleType createType() {
51         return new FamilyType();
52     }
53 }
54
```

```

55 class MotorcycleFactory extends AbstractFactory {
56     @Override
57     Model createModel() {
58         return new SportModel();
59     }
60
61     @Override
62     VehicleType createType() {
63         return new RacingType();
64     }
65 }
66
67
68 class VehicleClient {
69     private final Model model;
70     private final VehicleType vehicleType;
71
72     VehicleClient(AbstractFactory factory) {
73         this.model = factory.createModel();
74         this.vehicleType = factory.createType();
75     }
76
77     void displayInfo(String category) {
78         System.out.println("Category : " + category);
79         System.out.println("Model      : " + model.getModelName());
80         System.out.println("Type       : " + vehicleType.getTypeName());
81         System.out.println();
82     }
83 }
84
85 class Main {
86     public static void main(String[] args) {
87         AbstractFactory carFactory = new CarFactory();
88         VehicleClient car = new VehicleClient(carFactory);
89         car.displayInfo("Car");
90
91         AbstractFactory motorcycleFactory = new MotorcycleFactory();
92         VehicleClient motorcycle = new VehicleClient(motorcycleFactory);
93         motorcycle.displayInfo("Motorcycle");
94     }
95 }
96
97

```